RETAIL DATA AGENT · QUERY GATEWAY · AS OF 2026-11-01

What happens to the AI's SQL on its way to BigQuery?

It passes through four steps in one file, `gateway.py`. Some steps only check the SQL. Two of them rewrite it, so what BigQuery runs is safer than what the AI wrote. Below, one real question goes all the way through. See also every parser rule, brand scoping and the agent loop.

🟧 the AI 🟩 a check: only reads the SQL 🟪 a rewrite: changes the SQL 🟦 BigQuery 🟥 stopped here

In short

1. **Checks first.** Is it a safe, single read of our 4 tables, using only allowed columns, with age, city and the like only in totals of 5+ customers? If not, it stops, and the AI gets the reason so it can fix its SQL.
2. **Then two rewrites.** Customer ids become nicknames like `cust_249e6e6fc97d`, and every table is cut down to the user's brands.
3. **Then a free size check.** Anything that would scan more than 1 GB is stopped before it costs money.
4. **Only then BigQuery runs it.** The rows go back to the AI, which writes the answer.

One question, step by step

“Who are my top 3 customers?”

asked by the Levi's manager (`--user levis_manager`), who may see Levi's only

THE AI WRITES SQL `LLM → run_sql tool`

It writes the obvious query over the real tables. No brand filter, no nicknames: it doesn't have to think about safety.

```
SELECT u.id, COUNT(DISTINCT o.order_id) AS orders
FROM `…thelook_ecommerce.users` u
JOIN `…thelook_ecommerce.orders` o ON o.user_id = u.id
GROUP BY u.id
ORDER BY orders DESC
LIMIT 3
```

`gateway.run(sql)`

STEP 1 · GUARD `step1_guard.py`

🏠 Like a librarian at the door: one request slip, to *read*, from the shelves you're allowed.

✅ One statement, and it parses

✅ A SELECT, nothing that changes data

✅ No `@` variables (they could read our secret salt)

✅ Only real tables, and only our 4: `users`, `orders`

✕ **Stopped if** e.g. `DELETE FROM orders` → AI is told `Only SELECT queries are allowed.`

The SQL is unchanged.

STEP 2A · PERSONAL-DATA COLUMNS `step2_personal_data/columns.py`

🏠 Like a guest list: only columns written in `schema.yaml` get in. Names, email and addresses aren't on it.

✅ `id`, `order_id`, `user_id` are all on the list

✅ No `*`, no whole rows

✕ **Stopped if** e.g. `SELECT email …` → `Read only the columns listed in the schema.`

The SQL is unchanged.

STEP 2B · SMALL GROUPS `step2_personal_data/small_groups.py`

Age, gender, city, state, country and traffic source are tagged `aggregate_only` in `schema.yaml`. They may appear only inside a total, and the gateway adds `HAVING COUNT(DISTINCT …id) >= 5` to that total, so BigQuery drops groups under 5 customers.

🏠 Like a census office: counts per town, never a list of residents, and no town with fewer than 5 people.

✅ No tagged column in this query, so nothing to do

✕ **Stopped if** e.g. `SELECT id, city FROM users` → `` `users.city` can be used only inside a total `` …. Had the AI added `u.country` to this query, every group would be one customer, and the `HAVING` would hide every row.

The SQL is unchanged here. For a total of a tagged column, one `HAVING` line is added. See small groups.

STEP 2C · CUSTOMER PSEUDONYMS `step2_personal_data/customer_pseudonyms.py`

The customer tables are swapped for copies where each real id is replaced by a nickname. BigQuery makes the copies itself while it runs, so the AI's part of the query only ever sees nicknames. This happens for every user, the CEO too.

🏠 Like a coat check: you get ticket #72, never the hanger's location. Same coat, same ticket, every time.

```
WITH _pseudonymized_users AS (
  SELECT * REPLACE (CONCAT('cust_', …SHA256(@pseudonym_salt + id)…) AS id)  -- 48213 → cust_249e6e6fc97d
  FROM `…users` AS users),
_pseudonymized_orders AS ( …same for user_id… )
SELECT u.id, COUNT(DISTINCT o.order_id) AS orders
FROM _pseudonymized_users AS u
JOIN _pseudonymized_orders AS o ON o.user_id = u.id   -- nickname = nickname, so the join still matches
GROUP BY u.id ORDER BY orders DESC LIMIT 3
```

STEP 3 · BRAND SCOPE `step3_brand_scope/brand_scope.py`

Every real table, including the ones inside the nickname copies, is swapped for a copy that holds only Levi's rows. Customers and orders are kept only if they bought a Levi's product. The CEO sees every brand, so this step skips the CEO.

🏠 Like handing someone a filing cabinet that holds only their own folders.

```
WITH _brand_scoped_products AS (… WHERE brand IN ('Levi\'s')),
     _brand_scoped_order_items AS (… only Levi's products …),
     _brand_scoped_orders AS (… only orders with a Levi's item …),
     _brand_scoped_users  AS (… only people who bought Levi's …),
     _pseudonymized_users  AS (… FROM _brand_scoped_users …),
     _pseudonymized_orders AS (… FROM _brand_scoped_orders …)
SELECT … FROM _pseudonymized_users AS u JOIN _pseudonymized_orders AS o …
```

✕ **Stopped if** a table has no brand rule → `Table … can't be limited to your brands, so the query was not run.`

STEP 4 · SIZE CHECK `gateway.py · _check_bytes`

BigQuery is asked, for free, “how much would this read?” without running it (a *dry run*).

✅ 8.7 MB, under the 1 GB cap

✕ **Stopped if** over 1 GB → `This query would scan 1.4 GB, over the 1.0 GB limit per query. Select fewer columns or filter on a narrower date range.`

`database.execute(sql, max_rows=50)`

BIGQUERY RUNS IT `db/bigquery.py`

It receives the full rewritten SQL, plus the secret salt sent *alongside* it as `@pseudonym_salt`, never written into the SQL text or the log. At most 50 rows come back.

ID	ORDERS
cust_249e6e6fc97d	2
cust_267918d59c6e	2
cust_34fed0dbdb6b	2

Real rows from a run on 2026-11-01. Order counts are Levi's orders only. The nicknames change when the salt changes.

`rows → run_sql → back to the AI`

THE AI ANSWERS `LLM`

It reads the three rows and writes the answer in plain words. If the user follows up with “what did the first one buy?”, the AI writes `WHERE user_id = 'cust_249e6e6fc97d'`, and the whole journey runs again: the copies are rebuilt the same way, so the nickname matches.

What stops a query, and what the AI is told

A stopped query never reaches BigQuery. The AI gets the message as `Error: …`, fixes its SQL and tries again, within its 5 LLM calls per question.

STEP	THE AI WROTE	THE AI IS TOLD
1 · Guard	`SELECT 1; DROP TABLE orders`	Send one statement per query.
1 · Guard	`DELETE FROM orders`	Only SELECT queries are allowed.
1 · Guard	`SELECT @pseudonym_salt`	Query parameters and system variables (@name, ?, @@name) are not allowed. Write values into the SQL.
1 · Guard	`FROM …thelook_ecommerce.events`	Table `…events` is not allowed. Use only these tables: ...
2a · Columns	`SELECT email FROM users`	Read only the columns listed in the schema. ...
2a · Columns	`SELECT * FROM users`	Name the columns you need instead of using *. COUNT(*) is fine.
2b · Small groups	`SELECT id, city FROM users`	`users.city` can be used only inside a total (COUNT, SUM, AVG ... with GROUP BY) ...
2b · Small groups	`SELECT ARRAY_AGG(city) FROM users`	`users.city` can't be collected with ARRAY_AGG or STRING_AGG ...
3 · Brand scope	`FROM …inventory_items`	Table `…inventory_items` can't be limited to your brands, so the query was not run.
4 · Size	`a query over 1 GB`	This query would scan 1.4 GB, over the 1.0 GB limit per query. ...

Steps 2c and 3 never stop a normal query and never tell the AI anything: they quietly rewrite it. Step 2b stops listings, and quietly adds a `HAVING` to totals.

Limits on every query

1 GB

Scan cap

Checked with a free dry run, and capped again by BigQuery itself (`maximum_bytes_billed`).

50

Rows back

At most 50 rows reach the AI; it's told when rows were cut off.

0

Real customer ids

The AI only ever sees `cust_…` nicknames, made from a secret salt it can't read.

> **The rule in one sentence**
>
> The AI writes plain SQL, the gateway checks it, then rewrites it for privacy and brands, and only that rewritten SQL reaches BigQuery.

Code: `QueryGateway.run` in `src/retail_agent/query_safety/gateway.py` calls, in order, `QueryGuard.check` (`step1_guard.py`), `PersonalDataCheck.check` (`step2_personal_data/columns.py`), `SmallGroups.apply` (`step2_personal_data/small_groups.py`), `CustomerPseudonyms.apply` (`step2_personal_data/customer_pseudonyms.py`), `BrandScope.apply` (`step3_brand_scope/brand_scope.py`), `_check_bytes`, then `BigQueryDatabase.execute` (`src/retail_agent/db/bigquery.py`). Wired in `assemble` (`src/retail_agent/app.py`).

Settings: `MAX_BYTES_BILLED` (1 GB), `MAX_ROWS` (50), `PSEUDONYM_SALT`, `MIN_GROUP_SIZE` (5), `USERS_PATH`. The original and rewritten SQL are logged as `SQL:` and `Rewritten SQL:` in `logs/retail-agent.log`.

Tests: `tests/test_step1_guard.py`, `tests/test_step2_personal_data.py`, `tests/test_step2_small_groups.py`, `tests/test_step2_customer_pseudonyms.py`, `tests/test_step3_brand_scope.py`. Tickets: 08 (columns), 09 (pseudonyms), 10 (small groups).

See also SQL parser rules, brand scoping, order item count, SQL errors, architecture, small groups.